Linux 入門

aoki-taquan

2026 年 03 月 16 日

目次

1 はじめに 5
1.1 Linux とは 5
1.2 ディストリビューション 5
1.3 本書の対象と構成 6
2 インストール 7
2.1 システム要件 7
2.2 インストール方法 7
2.2.1 ISO イメージからのインストール 7
2.2.2 仮想マシンへのインストール 7
2.2.3 クラウド環境 7
2.2.4 WSL（Windows Subsystem for Linux） 7
2.3 インストール後の初期設定 8
2.3.1 システムの更新 8
2.3.2 タイムゾーンの設定 8
2.3.3 ホスト名の設定 8
2.3.4 SSH サーバーの設定 8
3 基本コマンド 9
3.1 シェルとターミナル 9
3.2 コマンドの基本構文 9
3.3 ファイル・ディレクトリ操作 9
3.3.1 現在地の確認と移動 9
3.3.2 ファイル・ディレクトリの一覧 9
3.3.3 ファイル・ディレクトリの作成・削除 10
3.3.4 コピー・移動・リネーム 10
3.4 ファイル内容の確認 10
3.5 テキスト検索・加工 11
3.6 パイプとリダイレクト 11
3.7 マニュアルとヘルプ 11
4 ファイルシステム 13
4.1 ディレクトリ構造 13
4.2 パスの種類 13
4.3 ファイルの種類 13
4.4 リンク 14
4.4.1 ハードリンク 14
4.4.2 シンボリックリンク（ソフトリンク） 14
4.5 ディスク管理 14
4.5.1 ディスク使用量の確認 14
4.5.2 マウントとアンマウント 14
4.6 ファイルの検索 15

5 ユーザーとパーミッション 16
5.1 ユーザーとグループ 16
5.1.1 ユーザーの管理 16
5.1.2 グループの管理 16
5.2 パーミッション 17
5.2.1 数値表記 17
5.2.2 パーミッションの変更 17
5.2.3 所有者の変更 17
5.3 sudo 18
5.4 特殊なパーミッション 18
6 パッケージ管理 19
6.1 パッケージ管理の概要 19
6.2 APT（Debian / Ubuntu 系） 19
6.2.1 パッケージの検索・情報確認 19
6.2.2 パッケージのインストール・削除 19
6.2.3 システムの更新 20
6.3 DNF（RHEL / Rocky Linux / Fedora 系） 20
6.4 リポジトリの管理 20
6.4.1 APT リポジトリの追加 20
6.4.2 DNF リポジトリの追加 21
6.5 Snap と flatpak 21
7 プロセス管理 22
7.1 プロセスとは 22
7.2 プロセスの確認 22
7.2.1 ps コマンドの出力の見方 22
7.3 シグナルとプロセスの制御 23
7.4 ジョブ管理 23
7.5 リソースの監視 24
7.6 cron によるタスクスケジューリング 24
8 ネットワーク 25
8.1 ネットワークの基本確認 25
8.2 接続テスト 25
8.3 ネットワーク設定 25
8.3.1 Netplan による設定（Ubuntu） 25
8.3.2 NetworkManager による設定 26
8.4 ポートとソケット 26
8.5 ファイアウォール 27
8.5.1 ufw（Ubuntu） 27
8.5.2 firewalld（RHEL / Rocky Linux） 27
8.6 データ転送 28
8.7 SSH 28
9 シェルスクリプト 29

9.1 シェルスクリプトとは 29
9.2 変数 29
9.2.1 特殊変数 29
9.3 条件分岐 30
9.3.1 if 文 30
9.3.2 条件式 30
9.3.3 case 文 30
9.4 ループ 31
9.4.1 for ループ 31
9.4.2 while ループ 31
9.5 関数 32
9.6 実践的なスクリプト例 32
9.6.1 バックアップスクリプト 32
9.6.2 ヘルスチェックスクリプト 33
10 サービス管理 34
10.1 systemd 34
10.2 サービスの操作 34
10.3 ユニットファイル 34
10.3.1 ユニットファイルの例 34
10.3.2 セクションの説明 35
10.4 ログ管理（journalctl） 35
10.5 ターゲット（ランレベル） 36
10.6 systemd timer 36

1 はじめに

1.1 Linux とは

Linux は、1991 年にリーナス・トーバルズによって開発が始まったオープンソースのオペレーティングシステム（OS）カーネルである。現在では、サーバー、デスクトップ、組み込みシステム、スーパーコンピュータ、クラウドインフラなど、あらゆる分野で利用されている。

Linux の特徴を以下にまとめる。

特徴	説明
オープンソース	ソースコードが公開されており、誰でも自由に利用・改変・再配布できる
安定性	長期間の連続稼働に耐える高い安定性を持つ
セキュリティ	パーミッションモデルや SELinux など堅牢なセキュリティ機構を備える
柔軟性	用途に応じてカスタマイズが容易で、軽量構成から大規模構成まで対応可能
コミュニティ	世界中の開発者が参加する活発なコミュニティにより継続的に改善されている

1.2 ディストリビューション

Linux カーネル単体では OS として機能しないため、各種ソフトウェアを組み合わせた「ディストリビューション」として配布されている。主要なディストリビューションを以下に示す。

ディストリビューション	系統	特徴
Ubuntu	Debian 系	初心者に優しく、デスクトップ・サーバー両方で広く利用される
Debian	Debian 系	安定性を重視し、長い歴史を持つ
CentOS Stream / RHEL	Red Hat 系	企業向けで長期サポートが充実
Rocky Linux / AlmaLinux	Red Hat 系	CentOS 後継として企業用途で採用が進む
Fedora	Red Hat 系	最新技術をいち早く取り入れるディストリビューション
Arch Linux	独立系	シンプルさを追求し、ユーザーが一から構築する
openSUSE	SUSE 系	YaST による管理ツールが充実

1.3 本書の対象と構成

本書はLinuxをこれから学ぶ方を対象とした入門書である。基本的なコマンド操作からファイルシステム、ユーザー管理、ネットワーク設定、シェルスクリプトまで、実務で必要となる知識を体系的に解説する。

主にUbuntu / Debian系を例に説明するが、Red Hat系での違いについても適宜補足する。

2 インストール

2.1 システム要件

Linux は比較的軽量な OS であるが、快適に利用するための推奨スペックを以下に示す。

項目	最小要件	推奨
CPU	1 GHz 以上	2 コア以上
メモリ	512 MB	2 GB 以上
ストレージ	10 GB	25 GB 以上
ネットワーク	—	インターネット接続

サーバー用途の場合は GUI 不要のため、より少ないリソースで動作する。

2.2 インストール方法

Linux をインストールする方法はいくつかある。

2.2.1 ISO イメージからのインストール

最も一般的な方法である。各ディストリビューションの公式サイトから ISO イメージをダウンロードし、USB メモリや DVD に書き込んで起動する。

```
# USBメモリへの書き込み例（Linux環境）
sudo dd if=ubuntu-24.04-live-server-amd64.iso of=/dev/sdb bs=4M status=progress
sync
```

2.2.2 仮想マシンへのインストール

学習目的であれば、仮想マシン上にインストールするのが手軽である。

- **VirtualBox**: 無料で利用できる仮想化ソフトウェア
- **VMware Workstation Player**: 個人利用は無料
- **Proxmox VE**: サーバー向け仮想化プラットフォーム
- **KVM / QEMU**: Linux 標準の仮想化技術

2.2.3 クラウド環境

AWS、GCP、Azure などのクラウドサービスでは、あらかじめ Linux がインストールされた仮想マシンをすぐに利用できる。

2.2.4 WSL（Windows Subsystem for Linux）

Windows 上で Linux 環境を利用する場合は WSL が便利である。

```
# WSLのインストール（PowerShell管理者権限）
wsl --install

# 特定のディストリビューションを指定
wsl --install -d Ubuntu-24.04
```

2.3 インストール後の初期設定

2.3.1 システムの更新

インストール直後はパッケージを最新の状態に更新する。

```
# Ubuntu / Debian系
sudo apt update && sudo apt upgrade -y

# RHEL / Rocky Linux系
sudo dnf update -y
```

2.3.2 タイムゾーンの設定

```
# 現在のタイムゾーンを確認
timedatectl

# 日本時間に設定
sudo timedatectl set-timezone Asia/Tokyo
```

2.3.3 ホスト名の設定

```
# 現在のホスト名を確認
hostnamectl

# ホスト名を変更
sudo hostnamectl set-hostname myserver
```

2.3.4 SSH サーバーの設定

リモートアクセスのために SSH サーバーを有効にする。

```
# SSHサーバーのインストールと起動
sudo apt install -y openssh-server
sudo systemctl enable --now ssh

# ファイアウォールでSSHを許可（ufwの場合）
sudo ufw allow ssh
sudo ufw enable
```

3 基本コマンド

3.1 シェルとターミナル

Linux ではシェル（shell）と呼ばれるコマンドラインインターフェースを通じてシステムを操作する。最も広く使われているシェルは Bash（Bourne Again Shell）である。

ターミナル（端末エミュレータ）はシェルにアクセスするためのアプリケーションで、GUI デスクトップ環境では `Ctrl + Alt + T` で起動できることが多い。

3.2 コマンドの基本構文

```
コマンド [オプション] [引数]
```

例えば `ls -la /home` は、`ls` がコマンド、`-la` がオプション、`/home` が引数である。

3.3 ファイル・ディレクトリ操作

3.3.1 現在地の確認と移動

```
# 現在のディレクトリを表示
pwd

# ディレクトリの移動
cd /var/log          # 絶対パスで移動
cd ..                # 一つ上のディレクトリへ
cd ~                 # ホームディレクトリへ
cd -                 # 直前のディレクトリへ
```

3.3.2 ファイル・ディレクトリの一覧

```
# 基本的な一覧表示
ls

# 詳細表示（パーミッション、所有者、サイズ、更新日時）
ls -l

# 隠しファイルを含む全ファイルを表示
ls -la

# 人が読みやすいサイズ表示
ls -lh
```

```
# ディレクトリのツリー表示
tree -L 2
```

3.3.3 ファイル・ディレクトリの作成・削除

```
# ディレクトリの作成
mkdir mydir
mkdir -p parent/child/grandchild   # 親ディレクトリも同時に作成

# ファイルの作成（空ファイル）
touch newfile.txt

# ファイルの削除
rm file.txt
rm -r mydir          # ディレクトリごと削除
rm -ri mydir         # 確認付きで削除

# ディレクトリの削除（空の場合のみ）
rmdir emptydir
```

3.3.4 コピー・移動・リネーム

```
# ファイルのコピー
cp source.txt dest.txt
cp -r srcdir/ destdir/     # ディレクトリのコピー

# ファイルの移動（リネームも同じコマンド）
mv oldname.txt newname.txt
mv file.txt /tmp/
```

3.4 ファイル内容の確認

```
# ファイル全体を表示
cat file.txt

# ページ単位で表示（スクロール可能）
less file.txt

# 先頭・末尾の表示
head -n 20 file.txt       # 先頭20行
tail -n 20 file.txt       # 末尾20行
tail -f /var/log/syslog   # リアルタイムで末尾を追跡
```

3.5 テキスト検索・加工

```
# ファイル内の文字列検索
grep "error" /var/log/syslog
grep -r "TODO" ./src/          # ディレクトリ内を再帰検索
grep -i "warning" file.txt     # 大文字小文字を無視
grep -n "pattern" file.txt     # 行番号を表示

# テキストの置換
sed 's/old/new/g' file.txt            # 標準出力に結果を表示
sed -i 's/old/new/g' file.txt         # ファイルを直接編集

# 列の切り出し
cut -d',' -f1,3 data.csv       # CSVの1列目と3列目を抽出

# テキストの並べ替え・重複排除
sort file.txt
sort -u file.txt               # 重複排除して並べ替え
uniq                           # 連続する重複行を削除

# 行数・単語数・バイト数のカウント
wc -l file.txt                 # 行数
wc -w file.txt                 # 単語数
```

3.6 パイプとリダイレクト

Linux の強力な特徴の一つがパイプ（`|`）とリダイレクトである。

```
# パイプ: コマンドの出力を次のコマンドの入力にする
cat access.log | grep "404" | wc -l

# リダイレクト: 出力をファイルに書き込む
echo "Hello" > output.txt      # 上書き
echo "World" >> output.txt     # 追記

# 標準エラー出力のリダイレクト
command 2> error.log           # エラーのみファイルへ
command > all.log 2>&1         # 標準出力とエラー両方をファイルへ

# /dev/null（出力を破棄）
command > /dev/null 2>&1
```

3.7 マニュアルとヘルプ

```
# マニュアルページの表示
man ls
```

```
man 5 passwd          # セクション5のpasswdを表示

# 簡易ヘルプ
ls --help

# コマンドの場所を確認
which python3
type ls
```

4 ファイルシステム

4.1 ディレクトリ構造

Linux のファイルシステムはルート（`/`）を頂点としたツリー構造になっている。FHS（Filesystem Hierarchy Standard）により、主要なディレクトリの用途が規定されている。

ディレクトリ	説明
`/`	ルートディレクトリ。すべてのファイルの起点
`/bin`	基本的なコマンド（`ls`、`cp` など）
`/sbin`	システム管理用コマンド（`fdisk`、`iptables` など）
`/etc`	システムの設定ファイル
`/home`	一般ユーザーのホームディレクトリ
`/root`	root ユーザーのホームディレクトリ
`/var`	ログ、キャッシュなど可変データ
`/tmp`	一時ファイル（再起動時に削除される場合がある）
`/usr`	ユーザー用プログラムやライブラリ
`/opt`	サードパーティ製ソフトウェア
`/dev`	デバイスファイル
`/proc`	カーネル・プロセス情報（仮想ファイルシステム）
`/sys`	カーネルのデバイス・ドライバ情報
`/mnt`	一時的なマウントポイント
`/media`	リムーバブルメディアのマウントポイント

4.2 パスの種類

- **絶対パス**: ルートから始まるフルパス（例: `/home/user/documents/file.txt`）
- **相対パス**: 現在のディレクトリからの相対位置（例: `../documents/file.txt`）

特殊なディレクトリ表記:
- `.` — 現在のディレクトリ
- `..` — 一つ上のディレクトリ
- `~` — ホームディレクトリ

4.3 ファイルの種類

Linux では「すべてはファイル」という設計思想がある。`ls -l` の先頭文字でファイルの種類がわかる。

記号	種類	例
`-`	通常のファイル	`/etc/passwd`

`d`	ディレクトリ	`/home/user`
`l`	シンボリックリンク	`/usr/bin/python3 -> python3.12`
`b`	ブロックデバイス	`/dev/sda`
`c`	キャラクタデバイス	`/dev/tty`
`p`	名前付きパイプ	プロセス間通信用
`s`	ソケット	`/var/run/docker.sock`

4.4 リンク

4.4.1 ハードリンク

同じ i ノードを指す別名である。元のファイルと完全に同等で、元ファイルを削除してもデータは残る。

```
ln original.txt hardlink.txt
ls -li original.txt hardlink.txt   # iノード番号が同じことを確認
```

4.4.2 シンボリックリンク（ソフトリンク）

ファイルのパスを指すショートカットである。別のファイルシステムへのリンクも可能。

```
ln -s /path/to/original.txt symlink.txt
ls -l symlink.txt    # -> で参照先が表示される
```

4.5 ディスク管理

4.5.1 ディスク使用量の確認

```
# ファイルシステムの使用状況
df -h

# ディレクトリごとの使用量
du -sh /var/log
du -sh /home/*

# 大きなファイルの検索
find / -type f -size +100M -exec ls -lh {} \; 2>/dev/null
```

4.5.2 マウントとアンマウント

Linux ではストレージデバイスをディレクトリツリーに「マウント」して利用する。

```
# デバイスの一覧表示
lsblk

# マウント
sudo mount /dev/sdb1 /mnt/usb

# アンマウント
sudo umount /mnt/usb

# 起動時の自動マウント設定
cat /etc/fstab
```

4.6 ファイルの検索

```
# ファイル名で検索
find /home -name "*.txt"
find / -name "nginx.conf" 2>/dev/null

# ファイルの種類で検索
find /var -type d -name "log"     # ディレクトリのみ
find /tmp -type f -mtime -1       # 1日以内に更新されたファイル

# 高速なファイル検索（インデックスベース）
sudo updatedb
locate nginx.conf
```

5 ユーザーとパーミッション

5.1 ユーザーとグループ

Linux はマルチユーザーシステムであり、各ユーザーには固有の UID（User ID）が割り当てられる。ユーザーは一つ以上のグループに所属する。

```
# 現在のユーザー情報を確認
id
whoami

# ユーザー一覧（/etc/passwdを確認）
cat /etc/passwd

# グループ一覧
cat /etc/group
```

5.1.1 ユーザーの管理

```
# ユーザーの作成
sudo useradd -m -s /bin/bash newuser    # ホームディレクトリ付き
sudo passwd newuser                      # パスワードの設定

# ユーザーの削除
sudo userdel -r olduser                  # ホームディレクトリも削除

# ユーザー情報の変更
sudo usermod -aG docker newuser          # グループへの追加
sudo usermod -s /bin/zsh newuser         # シェルの変更
```

5.1.2 グループの管理

```
# グループの作成
sudo groupadd developers

# グループへのユーザー追加
sudo gpasswd -a user1 developers

# グループからのユーザー削除
sudo gpasswd -d user1 developers
```

5.2 パーミッション

Linux のファイルパーミッションは、所有者（owner）、グループ（group）、その他（others）の 3 つのカテゴリに対して、読み取り（r）、書き込み（w）、実行（x）の権限を設定する。

```
-rwxr-xr-- 1 user group 4096 Mar 16 10:00 script.sh
|┬┬┬┬┬┬┬┬
| │   │   └── その他: 読み取りのみ（r--）
| │   └────── グループ: 読み取り+実行（r-x）
| └──────────── 所有者: 全権限（rwx）
└──────────────── ファイルタイプ（-: 通常ファイル）
```

5.2.1 数値表記

パーミッションは数値でも表現できる。

権限	記号	数値
読み取り	`r`	4
書き込み	`w`	2
実行	`x`	1
なし	`-`	0

例えば `rwxr-xr--` は `754` となる（7=4+2+1、5=4+0+1、4=4+0+0）。

5.2.2 パーミッションの変更

```
# 記号モード
chmod u+x script.sh          # 所有者に実行権限を追加
chmod g-w file.txt            # グループから書き込み権限を削除
chmod o=r file.txt            # その他を読み取りのみに設定
chmod a+r file.txt            # 全員に読み取り権限を追加

# 数値モード
chmod 755 script.sh           # rwxr-xr-x
chmod 644 file.txt            # rw-r--r--
chmod 600 secret.key          # rw-------

# 再帰的に変更
chmod -R 755 /var/www/html
```

5.2.3 所有者の変更

```
# 所有者の変更
sudo chown user file.txt
sudo chown user:group file.txt      # 所有者とグループを同時に変更
sudo chown -R user:group /var/www   # 再帰的に変更
```

5.3 sudo

`sudo`（superuser do）は、一般ユーザーが root 権限でコマンドを実行するための仕組みである。

```
# root権限でコマンドを実行
sudo apt update

# rootユーザーとしてシェルを起動
sudo -i

# 別のユーザーとしてコマンドを実行
sudo -u postgres psql
```

sudo を利用できるユーザーは `/etc/sudoers` で管理される。編集には `visudo` コマンドを使用する。

```
# sudoersの編集（構文チェック付き）
sudo visudo

# 例: userにsudo権限を付与（sudoersの記述）
# user ALL=(ALL:ALL) ALL
```

5.4 特殊なパーミッション

名前	数値	説明
SUID	4000	実行時にファイル所有者の権限で動作する（例: `passwd` コマンド）
SGID	2000	実行時にグループの権限で動作する。ディレクトリに設定すると新規ファイルが親のグループを継承
Sticky bit	1000	ディレクトリに設定すると、ファイルの所有者のみが削除可能（例: `/tmp`）

```
# SUIDの設定
sudo chmod u+s /usr/local/bin/myapp
# または
sudo chmod 4755 /usr/local/bin/myapp

# Sticky bitの設定
sudo chmod +t /shared
# または
sudo chmod 1777 /shared
```

6 パッケージ管理

6.1 パッケージ管理の概要

Linux では、ソフトウェアの導入・更新・削除を「パッケージマネージャ」で一元管理する。ディストリビューションによって使用するパッケージマネージャが異なる。

系統	パッケージ形式	パッケージマネージャ
Debian / Ubuntu	`.deb`	`apt`（`dpkg`）
RHEL / Rocky Linux / Fedora	`.rpm`	`dnf`（`rpm`）
Arch Linux	`.pkg.tar.zst`	`pacman`
openSUSE	`.rpm`	`zypper`

6.2 APT（Debian / Ubuntu 系）

APT（Advanced Package Tool）は Debian 系で標準のパッケージマネージャである。

6.2.1 パッケージの検索・情報確認

```
# パッケージリストの更新
sudo apt update

# パッケージの検索
apt search nginx

# パッケージの詳細情報
apt show nginx

# インストール済みパッケージの一覧
apt list --installed
```

6.2.2 パッケージのインストール・削除

```
# インストール
sudo apt install nginx

# 複数パッケージの同時インストール
sudo apt install nginx curl vim

# パッケージの削除
sudo apt remove nginx              # 設定ファイルは残る
sudo apt purge nginx               # 設定ファイルも削除
```

```
# 不要な依存パッケージの削除
sudo apt autoremove
```

6.2.3 システムの更新

```
# パッケージリストの更新 + アップグレード
sudo apt update && sudo apt upgrade -y

# ディストリビューションのアップグレード（カーネル等も含む）
sudo apt full-upgrade -y
```

6.3 DNF（RHEL / Rocky Linux / Fedora 系）

DNF は Red Hat 系で標準のパッケージマネージャである。

```
# パッケージの検索
dnf search nginx

# インストール
sudo dnf install nginx

# 削除
sudo dnf remove nginx

# システムの更新
sudo dnf update -y

# パッケージグループのインストール
sudo dnf groupinstall "Development Tools"
```

6.4 リポジトリの管理

パッケージはリポジトリ（パッケージの配布元）からダウンロードされる。公式リポジトリに含まれないソフトウェアを利用する場合は、サードパーティのリポジトリを追加する。

6.4.1 APT リポジトリの追加

```
# リポジトリの追加（例: Dockerの公式リポジトリ）
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
```

```
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
```

6.4.2 DNF リポジトリの追加

```
# EPELリポジトリの追加
sudo dnf install epel-release

# リポジトリの一覧表示
dnf repolist
```

6.5 Snap と flatpak

従来のパッケージマネージャに加え、ディストリビューション非依存のパッケージ形式も普及している。

形式	説明
Snap	Canonical（Ubuntu）が推進。`snap install` で導入。依存関係を自己完結型で管理
Flatpak	Red Hat 等が推進。デスクトップアプリ向け。サンドボックス環境で動作

```
# Snapの利用例
sudo snap install code --classic       # VS Code

# Flatpakの利用例
flatpak install flathub org.gimp.GIMP
```

7 プロセス管理

7.1 プロセスとは

プロセスとは、実行中のプログラムのインスタンスである。各プロセスには PID（Process ID）が割り当てられ、カーネルによって管理される。

7.2 プロセスの確認

```
# 現在のシェルのプロセスを表示
ps

# 全プロセスを詳細表示
ps aux

# 特定のプロセスを検索
ps aux | grep nginx

# プロセスツリーの表示
pstree

# リアルタイムのプロセスモニタ
top

# より高機能なモニタ（インストールが必要な場合あり）
htop
```

7.2.1 ps コマンドの出力の見方

```
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 169392 13284 ?        Ss   Mar15   0:02 /sbin/init
```

フィールド	説明
USER	プロセスの実行ユーザー
PID	プロセス ID
%CPU	CPU 使用率
%MEM	メモリ使用率
VSZ	仮想メモリサイズ
RSS	物理メモリ使用量
STAT	プロセスの状態（S:スリープ、R:実行中、Z:ゾンビ、T:停止）
COMMAND	実行コマンド

7.3 シグナルとプロセスの制御

シグナルはプロセスに送られる通知で、プロセスの動作を制御するために使用する。

```
# プロセスの終了
kill PID              # SIGTERM（正常終了要求）
kill -9 PID           # SIGKILL（強制終了）
kill -HUP PID         # SIGHUP（設定の再読み込み）

# プロセス名で終了
killall nginx
pkill -f "python app.py"
```

主要なシグナル:

シグナル	番号	説明
SIGHUP	1	ハングアップ。設定再読み込みに使われることが多い
SIGINT	2	`Ctrl+C` で送られる割り込みシグナル
SIGTERM	15	正常終了要求（デフォルト）
SIGKILL	9	強制終了（プロセスは無視できない）
SIGSTOP	19	プロセスの一時停止
SIGCONT	18	停止したプロセスの再開

7.4 ジョブ管理

シェルでは、フォアグラウンドとバックグラウンドでジョブを管理できる。

```
# バックグラウンドで実行
long_running_command &

# 実行中のジョブ一覧
jobs

# フォアグラウンドのジョブをバックグラウンドへ
# Ctrl+Z で一時停止してから
bg

# バックグラウンドのジョブをフォアグラウンドへ
fg %1

# ターミナル切断後も継続して実行
nohup long_running_command &
```

7.5 リソースの監視

```
# メモリ使用量
free -h

# CPU情報
lscpu
cat /proc/cpuinfo

# ディスクI/Oの監視
iostat -x 1

# システム全体の負荷
uptime

# 詳細なリソース監視
vmstat 1 5            # 1秒間隔で5回表示
```

7.6 cron によるタスクスケジューリング

cron は Linux 標準のタスクスケジューラであり、定期的なジョブの実行に使用する。

```
# 現在のcrontabを表示
crontab -l

# crontabを編集
crontab -e
```

crontab の書式:

```
分 時 日 月 曜日 コマンド
```

```
# 毎日午前3時にバックアップスクリプトを実行
0 3 * * * /home/user/backup.sh

# 5分ごとにヘルスチェック
*/5 * * * * /usr/local/bin/healthcheck.sh

# 毎週月曜の午前9時にレポートを送信
0 9 * * 1 /home/user/report.sh

# 毎月1日の午前0時にログをローテーション
0 0 1 * * /usr/sbin/logrotate /etc/logrotate.conf
```

systemd ベースのシステムでは systemd timer も利用できる。

8 ネットワーク

8.1 ネットワークの基本確認

```
# IPアドレスの確認
ip addr show
ip a                        # 省略形

# ルーティングテーブルの確認
ip route show

# DNSの確認
cat /etc/resolv.conf
resolvectl status           # systemd-resolved環境

# ネットワークインターフェースの状態
ip link show
```

8.2 接続テスト

```
# 疎通確認
ping -c 4 8.8.8.8
ping -c 4 google.com

# 経路の確認
traceroute google.com
tracepath google.com

# ポートへの接続テスト
nc -zv example.com 443
curl -I https://example.com

# DNS名前解決
nslookup example.com
dig example.com
host example.com
```

8.3 ネットワーク設定

8.3.1 Netplan による設定（Ubuntu）

Ubuntu 18.04 以降では Netplan がネットワーク設定の標準となっている。

```yaml
# /etc/netplan/01-config.yaml
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: true
    eth1:
      addresses:
        - 192.168.1.100/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

```bash
# 設定の適用
sudo netplan apply

# 設定のテスト（問題があれば自動ロールバック）
sudo netplan try
```

8.3.2 NetworkManager による設定

```bash
# 接続の一覧
nmcli connection show

# 静的IPの設定
nmcli connection modify eth0 ipv4.addresses 192.168.1.100/24
nmcli connection modify eth0 ipv4.gateway 192.168.1.1
nmcli connection modify eth0 ipv4.dns "8.8.8.8 8.8.4.4"
nmcli connection modify eth0 ipv4.method manual
nmcli connection up eth0
```

8.4 ポートとソケット

```bash
# 使用中のポートを確認
ss -tulnp

# 特定のポートを使用しているプロセスを確認
ss -tlnp | grep :80

# 旧来のコマンド（非推奨だが広く使われる）
netstat -tulnp
```

`ss` コマンドのオプション:

オプション	説明
`-t`	TCP ソケットを表示
`-u`	UDP ソケットを表示
`-l`	リスニング中のソケットのみ
`-n`	名前解決せず数値で表示
`-p`	ソケットを使用しているプロセスを表示

8.5 ファイアウォール

8.5.1 ufw（Ubuntu）

ufw（Uncomplicated Firewall）は iptables のフロントエンドで、簡単にファイアウォールを設定できる。

```
# ufwの有効化
sudo ufw enable

# 状態の確認
sudo ufw status verbose

# ルールの追加
sudo ufw allow 22/tcp              # SSHを許可
sudo ufw allow 80,443/tcp          # HTTP/HTTPSを許可
sudo ufw allow from 192.168.1.0/24 # 特定のネットワークを許可
sudo ufw deny 3306                 # MySQLポートを拒否

# ルールの削除
sudo ufw delete allow 80/tcp
```

8.5.2 firewalld（RHEL / Rocky Linux）

```
# 状態の確認
sudo firewall-cmd --state

# 許可サービスの確認
sudo firewall-cmd --list-all

# サービスの許可（恒久的）
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload

# ポートの許可
```

```
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload
```

8.6 データ転送

```
# ファイルのダウンロード
curl -O https://example.com/file.tar.gz
wget https://example.com/file.tar.gz

# リモートサーバーとのファイル転送（SCP）
scp file.txt user@remote:/tmp/
scp user@remote:/tmp/file.txt ./

# ディレクトリごと転送（rsync）
rsync -avz /local/dir/ user@remote:/remote/dir/
rsync -avz --delete /local/dir/ user@remote:/remote/dir/
```

8.7 SSH

SSH（Secure Shell）はリモートサーバーへの暗号化された接続を提供する。

```
# リモートサーバーへの接続
ssh user@192.168.1.100
ssh -p 2222 user@example.com        # ポート指定

# SSH鍵の生成
ssh-keygen -t ed25519 -C "user@example.com"

# 公開鍵をリモートサーバーに登録
ssh-copy-id user@remote

# SSH設定ファイル（~/.ssh/config）
# Host myserver
#     HostName 192.168.1.100
#     User admin
#     Port 2222
#     IdentityFile ~/.ssh/id_ed25519
```

上記の設定を行うと `ssh myserver` で接続できるようになる。

9 シェルスクリプト

9.1 シェルスクリプトとは

シェルスクリプトは、シェルコマンドをファイルにまとめて自動実行するプログラムである。繰り返し行う作業や複雑な手順を自動化するために活用される。

```
#!/bin/bash
# ファイル名: hello.sh
echo "Hello, World!"
```

```
# 実行権限の付与と実行
chmod +x hello.sh
./hello.sh
```

1 行目の `#!/bin/bash` はシバン（shebang）と呼ばれ、スクリプトを実行するインタプリタを指定する。

9.2 変数

```
#!/bin/bash

# 変数の定義（=の前後にスペースを入れない）
name="Linux"
version=6

# 変数の参照
echo "OS: $name"
echo "Kernel: ${version}.x"

# コマンドの実行結果を変数に格納
current_date=$(date '+%Y-%m-%d')
file_count=$(ls -1 | wc -l)

echo "日付: $current_date"
echo "ファイル数: $file_count"
```

9.2.1 特殊変数

変数	説明
`$0`	スクリプト名
`$1` 〜 `$9`	引数（1 番目〜9 番目）
`$#`	引数の数

`$@`	すべての引数（個別に展開）
`$?`	直前のコマンドの終了ステータス
`$$`	現在のプロセス ID

9.3 条件分岐

9.3.1 if 文

```
#!/bin/bash

if [ -f "/etc/nginx/nginx.conf" ]; then
    echo "Nginxの設定ファイルが存在します"
elif [ -f "/etc/apache2/apache2.conf" ]; then
    echo "Apacheの設定ファイルが存在します"
else
    echo "Webサーバーの設定ファイルが見つかりません"
fi
```

9.3.2 条件式

カテゴリ	条件式	説明
ファイル	`-f file`	通常ファイルが存在する
ファイル	`-d dir`	ディレクトリが存在する
ファイル	`-e path`	パスが存在する
ファイル	`-r file`	読み取り可能
ファイル	`-w file`	書き込み可能
ファイル	`-x file`	実行可能
文字列	`-z str`	文字列が空
文字列	`-n str`	文字列が空でない
文字列	`str1 = str2`	文字列が等しい
数値	`n1 -eq n2`	等しい
数値	`n1 -ne n2`	等しくない
数値	`n1 -gt n2`	より大きい
数値	`n1 -lt n2`	より小さい

9.3.3 case 文

```
#!/bin/bash

case "$1" in
    start)
```

```
        echo "サービスを開始します"
        ;;
    stop)
        echo "サービスを停止します"
        ;;
    restart)
        echo "サービスを再起動します"
        ;;
    *)
        echo "Usage: $0 {start|stop|restart}"
        exit 1
        ;;
esac
```

9.4 ループ

9.4.1 for ループ

```
#!/bin/bash

# リストの繰り返し
for fruit in apple banana orange; do
    echo "果物: $fruit"
done

# 数値の範囲
for i in {1..5}; do
    echo "番号: $i"
done

# ファイルの繰り返し
for file in /var/log/*.log; do
    echo "ログファイル: $file"
done
```

9.4.2 while ループ

```
#!/bin/bash

count=1
while [ $count -le 5 ]; do
    echo "カウント: $count"
    count=$((count + 1))
done

# ファイルを1行ずつ読み込む
```

```
while IFS= read -r line; do
    echo "行: $line"
done < /etc/hosts
```

9.5 関数

```
#!/bin/bash

# 関数の定義
check_service() {
    local service_name=$1
    if systemctl is-active --quiet "$service_name"; then
        echo "$service_name は稼働中です"
        return 0
    else
        echo "$service_name は停止しています"
        return 1
    fi
}

# 関数の呼び出し
check_service nginx
check_service mysql
```

9.6 実践的なスクリプト例

9.6.1 バックアップスクリプト

```
#!/bin/bash
set -euo pipefail

BACKUP_DIR="/backup"
SOURCE_DIR="/var/www"
DATE=$(date '+%Y%m%d_%H%M%S')
ARCHIVE="${BACKUP_DIR}/www_${DATE}.tar.gz"

# バックアップディレクトリの作成
mkdir -p "$BACKUP_DIR"

# アーカイブの作成
tar czf "$ARCHIVE" -C "$(dirname "$SOURCE_DIR")" "$(basename "$SOURCE_DIR")"

echo "バックアップ完了: $ARCHIVE"

# 7日以上前のバックアップを削除
```

```
find "$BACKUP_DIR" -name "www_*.tar.gz" -mtime +7 -delete
echo "古いバックアップを削除しました"
```

9.6.2 ヘルスチェックスクリプト

```
#!/bin/bash
set -euo pipefail

SERVICES=("nginx" "mysql" "redis-server")
ALERT_EMAIL="admin@example.com"

for service in "${SERVICES[@]}"; do
    if ! systemctl is-active --quiet "$service"; then
        echo "[ALERT] $service が停止しています" | \
            mail -s "サービス停止: $service" "$ALERT_EMAIL"
        systemctl restart "$service"
        echo "$service を再起動しました"
    fi
done
```

スクリプトの先頭に `set -euo pipefail` を記述することで、エラー発生時にスクリプトを即座に停止させることができる。これは堅牢なスクリプトを書く上での基本的な慣習である。

10 サービス管理

10.1 systemd

systemd は、現在の主要な Linux ディストリビューションで採用されているシステム・サービスマネージャである。サービスの起動・停止・監視をはじめ、ログ管理やタイマーなど多くの機能を提供する。

10.2 サービスの操作

```
# サービスの状態確認
systemctl status nginx

# サービスの起動・停止・再起動
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx

# 設定ファイルの再読み込み（プロセスは再起動しない）
sudo systemctl reload nginx

# 自動起動の有効化・無効化
sudo systemctl enable nginx          # 起動時に自動で開始
sudo systemctl disable nginx         # 自動起動を無効化
sudo systemctl enable --now nginx    # 有効化と同時に起動

# サービスの一覧
systemctl list-units --type=service
systemctl list-units --type=service --state=running
```

10.3 ユニットファイル

systemd のサービスは「ユニットファイル」で定義される。ユニットファイルは通常 `/etc/systemd/system/` に配置する。

10.3.1 ユニットファイルの例

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network.target
Wants=network-online.target

[Service]
```

```
Type=simple
User=appuser
Group=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

10.3.2 セクションの説明

セクション	キー	説明
`[Unit]`	`Description`	サービスの説明
`[Unit]`	`After`	このサービスより先に起動すべきユニット
`[Unit]`	`Wants`	弱い依存関係（なくても起動する）
`[Service]`	`Type`	サービスの起動タイプ（simple, forking, oneshot 等）
`[Service]`	`ExecStart`	起動コマンド
`[Service]`	`Restart`	再起動ポリシー（on-failure, always 等）
`[Service]`	`User`	実行ユーザー
`[Install]`	`WantedBy`	有効化時にどのターゲットに紐づけるか

```
# ユニットファイルの変更後はデーモンをリロード
sudo systemctl daemon-reload

# サービスを起動
sudo systemctl start myapp
sudo systemctl enable myapp
```

10.4 ログ管理（journalctl）

systemd のログは journald によって管理される。`journalctl` コマンドで閲覧する。

```
# 全ログの表示
journalctl

# 特定のサービスのログ
journalctl -u nginx
```

```
# リアルタイムでログを追跡
journalctl -u nginx -f

# 今日のログのみ
journalctl --since today

# 期間を指定
journalctl --since "2026-03-15" --until "2026-03-16"

# カーネルログ
journalctl -k

# ログの使用量確認
journalctl --disk-usage

# 古いログの削除
sudo journalctl --vacuum-time=7d     # 7日以上前のログを削除
sudo journalctl --vacuum-size=500M   # 500MB以下に削減
```

10.5 ターゲット（ランレベル）

systemd では従来のランレベルに代わり「ターゲット」でシステムの状態を管理する。

ターゲット	旧ランレベル	説明
`poweroff.target`	0	システム停止
`rescue.target`	1	シングルユーザーモード（レスキュー）
`multi-user.target`	3	マルチユーザー（GUI なし）
`graphical.target`	5	マルチユーザー（GUI 付き）
`reboot.target`	6	再起動

```
# 現在のターゲットを確認
systemctl get-default

# デフォルトのターゲットを変更
sudo systemctl set-default multi-user.target

# システムの再起動・停止
sudo systemctl reboot
sudo systemctl poweroff
```

10.6 systemd timer

cron の代替として、systemd timer を使用した定期実行も可能である。

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Daily backup timer

[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

```
# /etc/systemd/system/backup.service
[Unit]
Description=Backup service

[Service]
Type=oneshot
ExecStart=/home/user/backup.sh
```

```
# timerの有効化
sudo systemctl enable --now backup.timer

# timerの一覧
systemctl list-timers
```